

PROIECT 2 – APLICAȚII PRACTICE ALE MICROCONTROLERELOR

SISTEM DE MONITORIZARE A TRAFICULUI

Coordonator științific: Guzu Nicolae-Alexandru

Studenți: Cornilă Kevin-Andrei, Zaharia Andrei-Cristian, Niță Abid-Ionel

Grupa: 434D

CUPRINS

1. INTRODUCERE	3
1.1. Context și motivație.....	3
1.2. Scopul proiectului.....	3
1.3. Obiectivele proiectului	3
1.4. Funcționalitățile implementate	3
1.5. Structura generală a sistemului.....	4
2. RESURSE HARDWARE	5
2.1. Platforma hardware principală.....	5
2.2. Modulele de semaforizare	5
2.3. Senzorii IR utilizați.....	6
2.4. Interfața UART.....	6
2.5. Configurația pinilor GPIO și EXTI	7
2.6. Alimentarea sistemului	7
3. RESURSE SOFTWARE	8
3.1. Mediul de dezvoltare utilizat	8
3.2. STM32CubeMX	8
3.3. Biblioteca HAL	9
3.4. Limbajul de programare utilizat	9
3.5. Structura modulară a aplicației	10
4. IMPLEMENTARE HARDWARE.....	11
4.1. Arhitectura hardware a sistemului.....	11
4.2. Implementarea modulelor de semaforizare	11
4.3. Implementarea sistemului de detecție IR.....	12
4.4. Configurarea întreruperilor externe EXTI	13
4.5. Implementarea comunicației UART	13
4.6. Considerații privind alimentarea și stabilitatea sistemului	14
5. IMPLEMENTARE SOFTWARE	15
5.1. Arhitectura software a aplicației.....	15
5.2. Implementarea mașinii de stări finite	15
5.3. Implementarea controlului semafoarelor.....	15
5.4. Implementarea sistemului de detecție a traficului	16
5.5. Implementarea logicii adaptive	16
5.6. Implementarea mecanismului de prevenire a starvation-ului	16
5.7. Implementarea monitorizării UART	17
5.8. Fluxul general de execuție al aplicației	17
6. CONCLUZII.....	18
7. BIBLIOGRAFIE	19

1. INTRODUCERE

1.1. Context și motivație

În contextul dezvoltării sistemelor inteligente de transport, optimizarea traficului rutier reprezintă o problemă importantă atât în mediul urban, cât și în zonele cu trafic variabil. Sistemele clasice de semaforizare utilizează timpi fixi de funcționare, fără a lua în considerare densitatea traficului sau numărul de vehicule aflate în așteptare pe fiecare direcție. Această abordare conduce frecvent la timpi inutili de așteptare și la utilizarea ineficientă a infrastructurii rutiere.

În cadrul acestui proiect a fost realizat un sistem inteligent de control al traficului utilizând platforma STM32 și senzori de detecție IR. Sistemul propus implementează un mecanism adaptiv de semaforizare, capabil să modifice durata semnalului verde în funcție de densitatea traficului detectat în intersecție.

Proiectul urmărește simularea funcționării unui controler de trafic inteligent pentru o intersecție cu patru direcții, utilizând tehnici specifice sistemelor embedded: programare neblocantă, întreruperi externe, mașini de stări finite și monitorizare UART în timp real.

1.2. Scopul proiectului

Scopul principal al proiectului constă în dezvoltarea unui sistem embedded capabil să gestioneze adaptiv funcționarea unei intersecții semaforizate cu patru direcții, pe baza informațiilor obținute de la senzori de trafic. Sistemul trebuie să fie capabil să detecteze vehiculele aflate pe fiecare direcție, să estimeze existența cozilor de vehicule și să adapteze timpul de semnal verde astfel încât direcțiile cu trafic ridicat să beneficieze de un timp suplimentar de traversare. De asemenea, proiectul urmărește implementarea unei arhitecturi software modulare și scalabile, specifică aplicațiilor embedded moderne.

1.3. Obiectivele proiectului

Principalele obiective urmărite în cadrul proiectului sunt:

- realizarea unui sistem de semaforizare pentru o intersecție cu patru direcții;
- implementarea controlului semafoarelor utilizând o mașină de stări finite (FSM - Finite State Machine);
- utilizarea întreruperilor externe (EXTI) pentru detecția vehiculelor;
- implementarea mecanismelor de debounce software pentru eliminarea detecțiilor false;
- realizarea unei logici de estimare a densității traficului;
- implementarea unei logici adaptive pentru modificarea timpului de semnal verde;
- implementarea unui mecanism de detectare a cozilor de vehicule;
- implementarea unui mecanism de prevenire a „starvation-ului” pentru direcțiile mai puțin utilizate;
- realizarea unei interfețe UART pentru monitorizarea funcționării sistemului în timp real;
- dezvoltarea unei structuri software modulare, ușor de extins și întreținut.

1.4. Funcționalitățile implementate

Sistemul dezvoltat include următoarele funcționalități principale:

- controlul independent al semafoarelor pentru patru direcții;
- secvențierea automată a stărilor verde - galben - roșu;
- detectarea vehiculelor utilizând senzori IR;
- numărarea vehiculelor pe fiecare direcție;

- estimarea existenței cozilor de vehicule prin utilizarea a doi senzori pe fiecare direcție;
- adaptarea dinamică a timpului de semnal verde în funcție de trafic;
- acordarea unui timp suplimentar direcțiilor aglomerate;
- reducerea progresivă a timpului de verde în condiții de trafic redus;
- prevenirea „starvation-ului” prin acordarea unui bonus direcțiilor care au așteptat mai multe cicluri;
- monitorizarea parametrilor sistemului prin UART.

1.5. Structura generală a sistemului

Sistemul este alcătuit dintr-o unitate centrală de control bazată pe microcontroller STM32, patru module de semaforizare și senzori IR utilizați pentru detectarea traficului. Fiecare direcție a intersecției utilizează doi senzori:

- un senzor poziționat la distanță mai mare față de intersecție (FAR), utilizat pentru estimarea fluxului de trafic;
- un senzor poziționat în apropierea intersecției (NEAR), utilizat pentru detectarea vehiculelor aflate în așteptare.

Informațiile colectate de la senzori sunt procesate de controlerul STM32, care adaptează durata semnalelor de circulație în funcție de condițiile de trafic detectate. Monitorizarea funcționării sistemului este realizată prin intermediul unei interfețe UART, utilizată pentru afișarea în timp real a parametrilor principali ai sistemului.

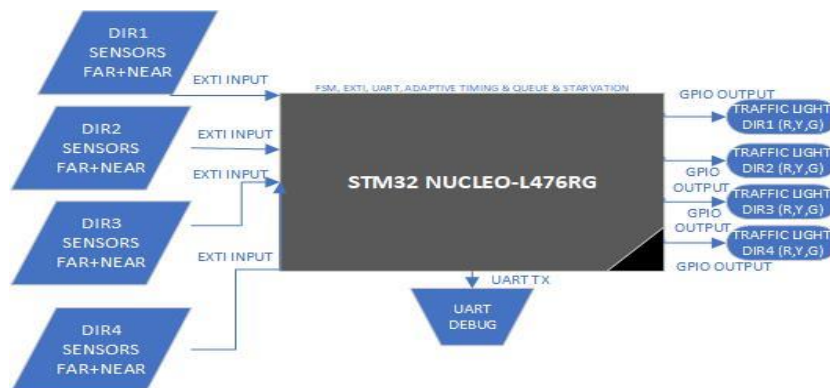


Fig. 1.1 - Diagrama generală a sistemului

2. RESURSE HARDWARE

2.1. Platforma hardware principală

Platforma hardware utilizată pentru implementarea sistemului este reprezentată de placa de dezvoltare STM32 NUCLEO-L476RG, bazată pe microcontrolerul STM32L476RG produs de STMicroelectronics. Microcontrolerul STM32L476RG utilizează un nucleu ARM Cortex-M4 pe 32 de biți și oferă suport pentru multiple periferice hardware necesare implementării sistemelor embedded moderne, precum:

- GPIO configurabile;
- întreruperi externe (EXTI);
- interfețe UART;
- timere hardware;
- suport pentru programare și depanare prin ST-LINK.
- Placa NUCLEO-L476RG a fost aleasă datorită:
- numărului mare de pini GPIO disponibili;
- suportului oferit de STM32CubeIDE și STM32CubeMX;
- posibilității configurării rapide a întreruperilor externe;
- consumului redus de energie;
- documentației extinse disponibile.

În cadrul proiectului, microcontrolerul are rolul de unitate centrală de control, gestionând:

- secvențierea semafoarelor;
- citirea senzorilor;
- procesarea logicii adaptive;
- monitorizarea sistemului prin UART.

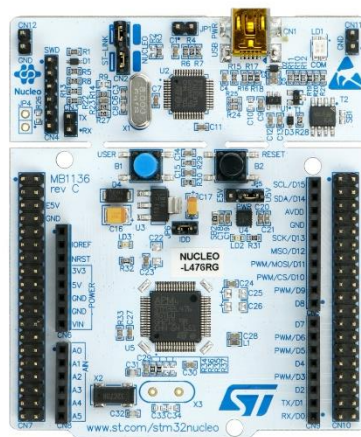


Fig. 2.1 - Placa de dezvoltare STM32 NUCLEO-L476RG [1]

2.2. Modulele de semaforizare

Sistemul utilizează patru module de semaforizare, corespunzătoare celor patru direcții ale intersecției.

Fiecare modul este alcătuit din trei LED-uri:

- LED roșu;
- LED galben;
- LED verde.

LED-urile sunt comandate direct de microcontroler prin intermediul pinilor GPIO configurați în modul Output Push-Pull.

Pentru limitarea curentului prin LED-uri au fost utilizate rezistențe de limitare conectate în serie cu fiecare LED.

Funcționarea semafoarelor respectă succesiunea clasică:

- verde;
- galben;
- roșu.

La un moment dat, o singură direcție poate avea semnal verde activ, celelalte direcții fiind menținute în stare roșie pentru evitarea conflictelor în intersecție.

2.3. Senzorii IR utilizați

Pentru detectarea vehiculelor au fost utilizați senzori IR de proximitate.

Fiecare direcție a intersecției utilizează doi senzori:

- un senzor FAR, poziționat la distanță mai mare față de intersecție;
- un senzor NEAR, poziționat în apropierea intersecției.

Această structură permite:

- estimarea fluxului de trafic;
- detectarea cozilor de vehicule;
- implementarea algoritmului adaptiv de control al traficului.
- ieșirile senzorilor sunt conectate la pini GPIO configurați cu întreruperi externe (EXTI), permițând detectarea rapidă a modificării stării senzorilor fără utilizarea polling-ului continuu.

Senzorii au fost configurați utilizând: rezistențe interne Pull-Up, detecție pe front crescător și descrescător (Rising/Falling Edge).

Semnalul activ al senzorilor este reprezentat de nivel logic LOW, ceea ce indică prezența unui obiect detectat.

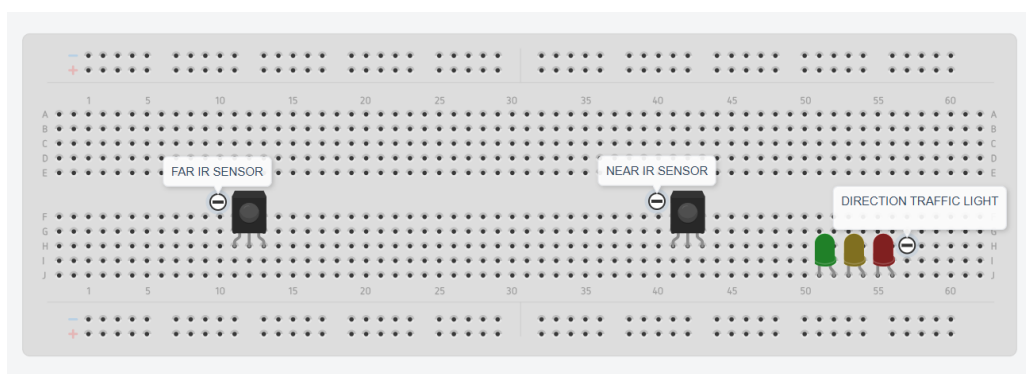


Fig. 2.2 - Poziționarea senzorilor FAR și NEAR pentru o direcție a intersecției

2.4. Interfața UART

Pentru monitorizarea funcționării sistemului a fost utilizată interfața UART disponibilă pe microcontrolerul STM32.

Comunicația serială este realizată prin perifericul USART2, configurat în modul asincron la viteza de 115200 baud.

Interfața UART este utilizată pentru:

- afișarea direcției active;
- afișarea timpului rămas pentru semnalul verde;
- monitorizarea numărului de vehicule detectate;
- afișarea informațiilor referitoare la cozi și trafic.

Transmiterea datelor este realizată către un terminal serial de pe calculator prin intermediul interfeței ST-LINK Virtual COM Port integrată pe placa NUCLEO.

2.5. Configurația pinilor GPIO și EXTI

Pinii GPIO utilizați pentru controlul semafoarelor au fost configurați în modul Output Push-Pull, fără rezistențe interne Pull-Up sau Pull-Down și cu viteză redusă de comutație (Low Speed).

Senzorii IR au fost conectați la pini configurați ca intrări cu întreruperi externe (GPIO_EXTI), utilizând rezistențe interne Pull-Up și detecție pe front crescător și descrescător.

Tabelul următor prezintă configurația principalilor pini utilizați în proiect.

FUNCȚIE	PIN STM32	CONFIGURAȚIE
RED LED D1	PB0	Output Push-Pull
YELLOW LED D1	PB1	Output Push-Pull
GREEN LED D1	PB2	Output Push-Pull
FAR SENSOR D1	PA8	GPIO_EXTI
NEAR SENSOR D1	PA9	GPIO_EXTI
RED LED D2	PB10	Output Push-Pull
YELLOW LED D2	PB11	Output Push-Pull
GREEN LED D2	PB12	Output Push-Pull
FAR SENSOR D2	PA10	GPIO_EXTI
NEAR SENSOR D2	PA11	GPIO_EXTI
RED LED D3	PB13	Output Push-Pull
YELLOW LED D3	PB14	Output Push-Pull
GREEN LED D3	PB15	Output Push-Pull
FAR SENSOR D3	PA12	GPIO_EXTI
NEAR SENSOR D3	PA15	GPIO_EXTI
RED LED D4	PC0	Output Push-Pull
YELLOW LED D4	PC1	Output Push-Pull
GREEN LED D4	PC2	Output Push-Pull
FAR SENSOR D4	PC6	GPIO_EXTI
NEAR SENSOR D4	PC7	GPIO_EXTI

Tabel 2.1 - Configurația principalilor pini GPIO utilizați

2.6. Alimentarea sistemului

Alimentarea sistemului este realizată prin conexiunea USB a plăcii STM32 NUCLEO-L476RG. Tensiunea de alimentare utilizată pentru logica sistemului și pentru periferice este de 3.3 V. LED-urile și senzorii IR sunt alimentați direct de la placa de dezvoltare, fără utilizarea unei surse externe suplimentare. Consumul redus de energie al componentelor utilizate permite funcționarea stabilă a sistemului fără necesitatea unor circuite suplimentare de alimentare.

3. RESURSE SOFTWARE

3.1. Mediul de dezvoltare utilizat

Pentru dezvoltarea aplicației software a fost utilizat mediul de dezvoltare integrat STM32CubeIDE, furnizat de compania STMicroelectronics.

STM32CubeIDE oferă suport complet pentru:

- configurarea perifericelor hardware;
- scrierea și compilarea codului sursă;
- depanarea aplicațiilor embedded;
- programarea microcontrolerului prin interfața ST-LINK.

Mediul de dezvoltare integrează atât editorul de cod, cât și utilitarul STM32CubeMX pentru configurarea perifericelor hardware și generarea automată a codului de inițializare.

Utilizarea STM32CubeIDE a permis dezvoltarea rapidă a aplicației și integrarea facilă a tuturor modulelor software utilizate în cadrul proiectului.

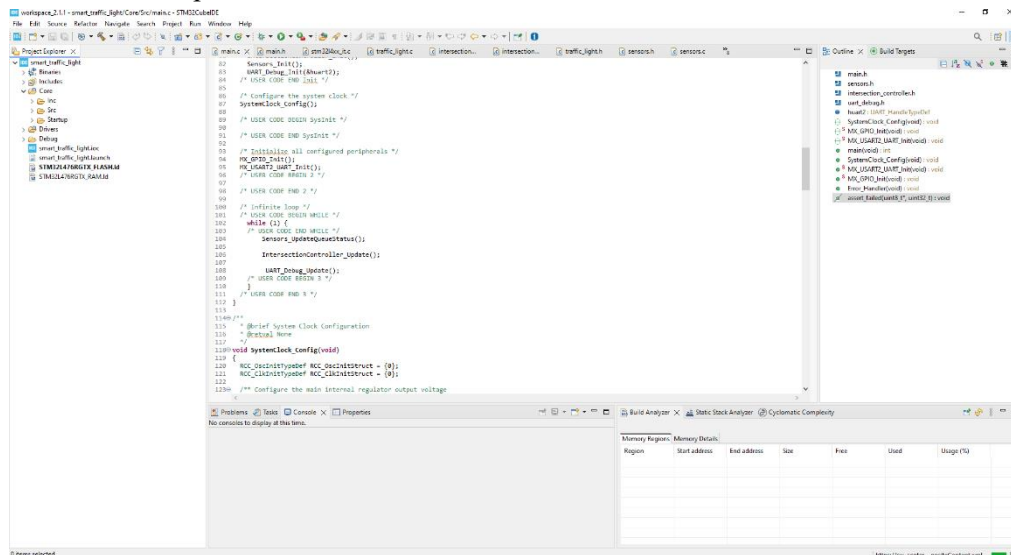


Fig. 3.1 - Mediul de dezvoltare STM32CubeIDE

3.2. STM32CubeMX

Configurarea perifericelor hardware ale microcontrolerului a fost realizată utilizând utilitarul STM32CubeMX integrat în STM32CubeIDE.

Acesta a fost utilizat pentru:

- configurarea pinilor GPIO;
- configurarea întreruperilor externe EXTI;
- configurarea interfeței UART;
- configurarea modurilor de funcționare pentru pini;
- generarea automată a codului de inițializare.

Pinii utilizați pentru semafoare au fost configurați în modul Output Push-Pull, iar pinii conectați la senzori au fost configurați ca intrări cu întreruperi externe și rezistențe interne Pull-Up.

Utilizarea STM32CubeMX a redus semnificativ timpul necesar configurării hardware și a permis evitarea erorilor de configurare manuală.



Fig. 3.2 - Configurația pinilor în STM32CubeMX

3.3. Biblioteca HAL

Pentru accesul la perifericele hardware ale microcontrolerului a fost utilizată biblioteca HAL (Hardware Abstraction Layer) oferită de STMicroelectronics.

Biblioteca HAL oferă funcții de nivel înalt pentru controlul perifericelor și simplifică dezvoltarea aplicațiilor embedded.

În cadrul proiectului au fost utilizate în principal următoarele funcții HAL:

- control GPIO;
- citirea stării pinilor;
- gestionarea întreruperilor externe;
- comunicația UART;
- gestionarea timpului prin funcția HAL_GetTick().

Utilizarea bibliotecii HAL a permis implementarea rapidă și portabilă a logicii aplicației, fără acces direct la registrele hardware ale microcontrolerului.

3.4. Limbajul de programare utilizat

Implementarea software a fost realizată utilizând limbajul de programare C.

Acesta este unul dintre cele mai utilizate limbaje în domeniul sistemelor embedded datorită:

- eficienței ridicate;
- controlului direct asupra resurselor hardware;
- consumului redus de memorie;
- portabilității ridicate.

Codul aplicației a fost dezvoltat utilizând o structură modulară, fiecare funcționalitate principală fiind implementată într-un fișier separat.

3.5. Structura modulară a aplicației

Aplicația software a fost organizată modular pentru a facilita:

- întreținerea codului;
- extinderea funcționalităților;
- separarea responsabilităților între componentele sistemului.
- Structura principală a aplicației include următoarele module:
- modul pentru controlul semafoarelor;
- modul pentru gestionarea senzorilor;
- modul pentru logica intersecției;
- modul pentru monitorizarea UART.

Separarea aplicației în module independente a permis dezvoltarea și testarea individuală a fiecărei componente software.

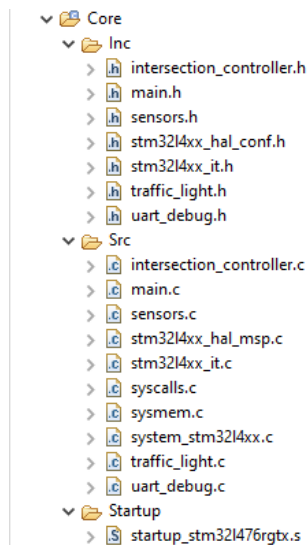


Fig. 3.3 - Structura modulară a aplicației software

4. IMPLEMENTARE HARDWARE

4.1. Arhitectura hardware a sistemului

Implementarea hardware a sistemului a fost realizată în jurul microcontrolerului STM32L476RG, utilizat ca unitate centrală de control pentru gestionarea semafoarelor și procesarea informațiilor provenite de la senzori.

Sistemul hardware este alcătuit din:

- placa de dezvoltare STM32 NUCLEO-L476RG;
- patru module de semaforizare;
- opt senzori IR de detecție;
- interfața UART pentru monitorizare;
- rezistențe de limitare pentru LED-uri.

Arhitectura implementată permite controlul independent al fiecărei direcții și monitorizarea în timp real a traficului detectat în intersecție.

Toate conexiunile hardware au fost realizate utilizând semnale digitale compatibile cu logica de 3.3 V a microcontrolerului STM32.

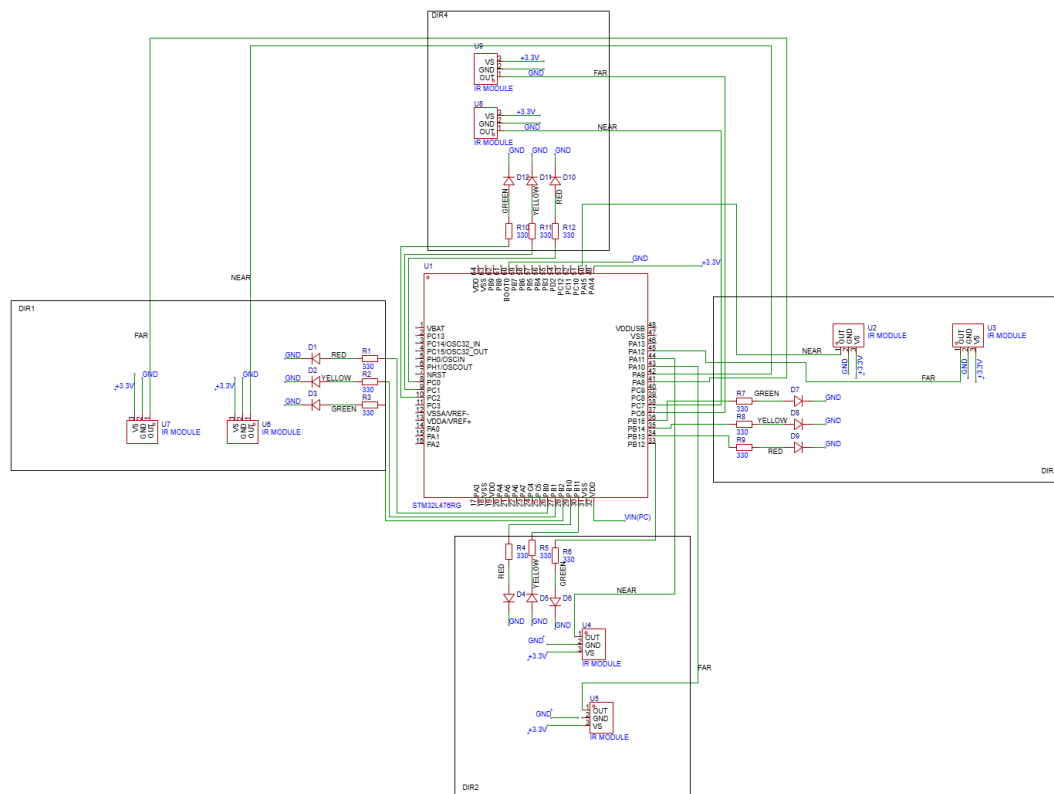


Fig. 4.1 - Schema generală hardware a sistemului

4.2. Implementarea modulelor de semaforizare

Fiecare direcție a intersecției utilizează trei LED-uri pentru simularea semnalelor unui semafor rutier:

- roșu;
- galben;
- verde.

LED-urile sunt conectate direct la pinii GPIO ai microcontrolerului prin intermediul unor rezistențe de limitare a curentului.

Pinii GPIO au fost configurați în modul Output Push-Pull pentru a permite controlul direct al stării fiecărui LED.

Pentru limitarea curentului prin LED-uri au fost utilizate rezistențe cu valori de 330 Ω . Alegerea acestor valori a fost realizată astfel încât curentul prin LED să rămână în limite sigure pentru GPIO-urile microcontrolerului.

Curentul aproximativ printr-un LED poate fi estimat utilizând relația:

$$I = \frac{V_{CC} - V_F}{R}$$

unde:

- V_{CC} reprezintă tensiunea de alimentare;
- V_F reprezintă tensiunea directă a LED-ului;
- R reprezintă rezistența de limitare.

Pentru:

- $V_{CC} = 3.3V$,
- $V_F \approx 2V$,
- $R = 330\Omega$,

rezultă un curent aproximativ de 4 mA, suficient pentru funcționarea corectă a LED-urilor și sigur pentru microcontroler.

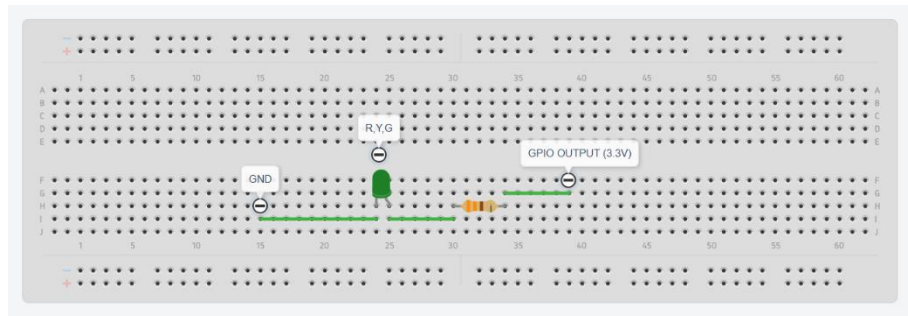


Fig. 4.2 - Schema conexiunii semafor

4.3. Implementarea sistemului de detecție IR

Fiecare direcție utilizează doi senzori IR:

- un senzor FAR;
- un senzor NEAR.

Senzorii sunt conectați la pini GPIO configurați cu întreruperi externe EXTI, ceea ce permite detectarea rapidă a modificării stării semnalului fără utilizarea unei metode de polling continuu.

Ieșirea senzorilor IR este activă în nivel logic LOW atunci când un obiect este detectat.

Pentru creșterea fiabilității sistemului au fost activate rezistențele interne Pull-Up ale microcontrolerului, astfel încât semnalul să rămână stabil în absența unui obiect detectat.

Configurația utilizată pentru senzori este:

- GPIO_EXTI;
- Pull-Up;
- Rising/Falling Edge Trigger.

Utilizarea a doi senzori pentru fiecare direcție permite estimarea densității traficului și detectarea existenței cozilor de vehicule.

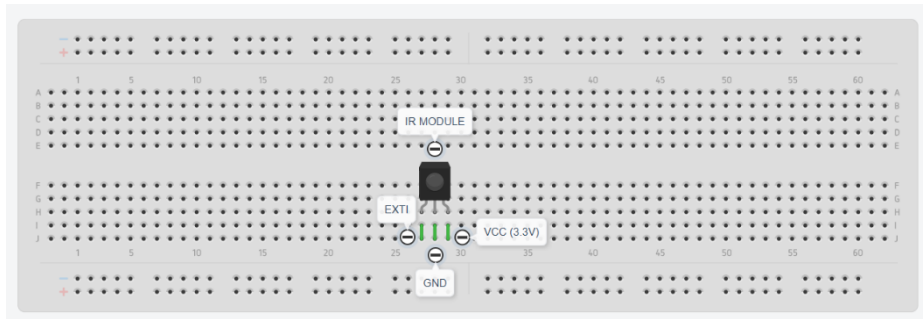


Fig. 4.3 - Schema conexiunii senzorilor IR

4.4. Configurarea întreruperilor externe EXTI

Detectarea vehiculelor este realizată utilizând întreruperile externe EXTI oferite de microcontrolerul STM32. Utilizarea întreruperilor externe prezintă avantajul reducerii încărcării procesorului, deoarece senzorii sunt procesați doar în momentul modificării stării semnalului.

În cadrul proiectului au fost utilizate două linii principale de întrerupere: EXTI9_5; EXTI15_10.

Acestea gestionează întreruperile generate de senzorii conectați pe liniile GPIO corespunzătoare.

Pentru eliminarea detecțiilor false generate de zgomot sau oscilații ale semnalului, a fost implementat un mecanism software de debounce bazat pe funcția HAL_GetTick().

Timpul de debounce utilizat în aplicație este de 100 ms.

4.5. Implementarea comunicației UART

Pentru monitorizarea funcționării sistemului a fost implementată o comunicație serială UART utilizând perifericul USART2 al microcontrolerului STM32.

Comunicația este configurată în modul asincron cu următorii parametri:

- baudrate: 115200;
- 8 biți de date;
- fără paritate;
- 1 bit de stop.

Datele sunt transmise către calculator prin intermediul interfeței ST-LINK Virtual COM Port integrate pe placa NUCLEO.

Interfața UART este utilizată pentru:

- afișarea stării sistemului;
- monitorizarea traficului;
- afișarea timpului de semnal verde rămas;

- depanarea aplicației.

4.6. Considerații privind alimentarea și stabilitatea sistemului

Întregul sistem este alimentat prin conexiunea USB a plăcii STM32 NUCLEO-L476RG.

Tensiunea de funcționare utilizată este de 3.3 V, compatibilă atât cu logica microcontrolerului, cât și cu senzorii IR utilizați.

Pentru creșterea stabilității sistemului au fost utilizate:

- rezistențe de limitare pentru LED-uri;
- rezistențe interne Pull-Up pentru senzori;
- debounce software pentru filtrarea semnalelor instabile.

De asemenea, utilizarea unei arhitecturi software neblocaante contribuie la funcționarea stabilă și predictibilă a întregului sistem embedded.

5. IMPLEMENTARE SOFTWARE

5.1. Arhitectura software a aplicației

Aplicația software a fost implementată utilizând o arhitectură modulară, fiecare funcționalitate principală fiind separată într-un modul software independent. Această abordare permite dezvoltarea și testarea individuală a componentelor sistemului și facilitează extinderea ulterioară a aplicației.

Structura aplicației include următoarele module principale: modulul de control al semafoarelor, modulul de gestionare a senzorilor, modulul principal de control al intersecției și modulul de monitorizare UART.

Fișierul `main.c` reprezintă punctul central de execuție al aplicației și are rolul de a inițializa perifericele hardware și de a executa bucla principală a sistemului. Modulul `traffic_light.c` gestionează controlul LED-urilor asociate semafoarelor, modulul `sensors.c` gestionează detecția vehiculelor și întreruperile externe, iar modulul `intersection_controller.c` implementează logica principală de control adaptiv al traficului.

Separarea funcționalităților în module distincte contribuie la creșterea lizibilității codului și la reducerea complexității generale a aplicației.

5.2. Implementarea mașinii de stări finite

Controlul intersecției a fost implementat utilizând o mașină de stări finite (FSM - Finite State Machine). Această abordare este frecvent utilizată în sistemele embedded deoarece permite controlul predictibil și clar al secvențelor de execuție.

Sistemul utilizează trei stări principale: stare de semnal verde, stare de semnal galben și stare de siguranță all-red. Tranzițiile dintre stări sunt realizate pe baza timpului măsurat utilizând funcția `HAL_GetTick()`.

Ordinea de funcționare a direcțiilor este circulară, fiecare direcție primind succesiv semnal verde. În timpul funcționării unei direcții, toate celelalte direcții sunt menținute în stare roșie pentru prevenirea conflictelor în intersecție.

Utilizarea unei implementări neblocaante bazate pe comparații temporale elimină necesitatea utilizării funcțiilor de tip `HAL_Delay()`, ceea ce permite funcționarea simultană a tuturor modulelor software și procesarea rapidă a întreruperilor externe.

Starea curentă a sistemului este actualizată periodic în funcția `IntersectionController_Update()`, apelată continuu în bucla principală a aplicației.

5.3. Implementarea controlului semafoarelor

Controlul semafoarelor a fost implementat în modulul `traffic_light.c`. Acesta gestionează activarea și dezactivarea LED-urilor corespunzătoare fiecărei direcții. Pentru fiecare direcție sunt controlate trei semnale: roșu, galben și verde. Pinii GPIO asociați LED-urilor sunt configurați în modul Output Push-Pull și sunt controlați utilizând funcțiile HAL pentru accesarea perifericelor GPIO.

În cadrul aplicației a fost implementată funcția `TrafficLight_SetState()`, responsabilă pentru modificarea stării unei direcții. De asemenea, funcția `TrafficLight_AllRed()` permite aducerea tuturor direcțiilor în stare de siguranță înaintea schimbării direcției active.

Separarea logicii de control a semafoarelor într-un modul dedicat permite reutilizarea și întreținerea facilă a codului sursă.

5.4. Implementarea sistemului de detecție a traficului

Detecția vehiculelor este realizată utilizând senzori IR conectați la pini GPIO configurați cu întreruperi externe EXTI. Fiecare direcție utilizează doi senzori: un senzor FAR și un senzor NEAR.

Modulul sensors.c gestionează atât citirea senzorilor, cât și procesarea întreruperilor generate de aceștia. În momentul activării unui senzor, funcția HAL_GPIO_EXTI_Callback() este executată automat și actualizează informațiile asociate direcției respective.

Pentru fiecare direcție sunt memorate următoarele informații: numărul total de activări ale senzorului FAR, numărul total de activări ale senzorului NEAR, starea curentă a senzorilor și existența unei cozi de vehicule.

Detectarea cozilor este realizată prin verificarea activării simultane a celor doi senzori ai unei direcții. Dacă atât senzorul FAR, cât și senzorul NEAR sunt activi simultan, sistemul consideră că există o acumulare de vehicule între cele două puncte de detecție.

Pentru eliminarea activărilor false a fost implementat un mecanism software de debounce bazat pe timpul furnizat de funcția HAL_GetTick().



Fig. 5.1 - Fluxul procesării întreruperilor

5.5. Implementarea logicii adaptive

Funcționalitatea principală a proiectului este reprezentată de implementarea logicii adaptive pentru controlul traficului. Durata semnalului verde este modificată dinamic în funcție de traficul detectat pe fiecare direcție.

Pentru fiecare ciclu de funcționare sunt analizate informațiile colectate de la senzori, precum numărul de vehicule detectate și existența cozilor de vehicule. Dacă traficul detectat depășește un prag prestabilit, timpul de semnal verde este mărit gradual pentru direcția respectivă.

Creșterea timpului de semnal verde este realizată incremental, utilizând pași de timp prestabiliți. De asemenea, timpul maxim de verde este limitat pentru prevenirea monopolizării intersecției de către o singură direcție.

În condiții de trafic redus, sistemul reduce progresiv timpul de verde până la o valoare minimă prestabilită. Această abordare permite adaptarea continuă a sistemului la variațiile traficului.

Implementarea logicii adaptive este realizată în cadrul modulului intersection_controller.c.



Fig. 5.2 - Fluxul algoritmului adaptiv

5.6. Implementarea mecanismului de prevenire a starvation-ului

Pentru evitarea situațiilor în care o direcție primește un timp insuficient de traversare pentru o perioadă lungă de timp, a fost implementat un mecanism de prevenire a starvation-ului.

Sistemul monitorizează numărul de cicluri în care fiecare direcție a rămas în stare roșie. Dacă o direcție depășește un prag prestabilit de așteptare, aceasta primește un bonus suplimentar de timp verde la următoarea activare.

Această abordare contribuie la menținerea unui comportament echilibrat al intersecției și împiedică favorizarea excesivă a direcțiilor foarte aglomerate.

Implementarea mecanismului de starvation prevention crește realismul și robustețea sistemului de control al traficului.

5.7. Implementarea monitorizării UART

Pentru monitorizarea funcționării sistemului a fost implementat un modul dedicat comunicației UART. Acesta permite afișarea în timp real a principalelor informații asociate sistemului. Datele transmise includ: direcția activă, timpul rămas pentru semnalul verde, valorile contoarelor senzorilor și starea cozilor de vehicule. Transmiterea datelor este realizată periodic prin intermediul perifericului USART2, utilizând funcția HAL_UART_Transmit(). Monitorizarea UART facilitează depanarea aplicației și permite verificarea funcționării corecte a algoritmului adaptiv în timp real.

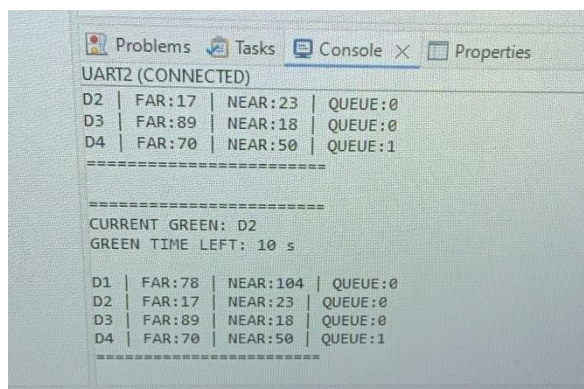


Fig. 5.3 - Monitorizare UART în terminal

5.8. Fluxul general de execuție al aplicației

După inițializarea perifericelor hardware, aplicația intră într-o buclă principală infinită în care sunt executate periodic toate funcțiile principale ale sistemului: System Init - while(1) - Sensors_UpdateQueueStatus() - IntersectionController_Update() - UART_Debug_Update() - Repeat.

Bucula principală actualizează starea cozilor de vehicule, execută logica mașinii de stări finite și transmite informațiile de monitorizare prin UART.

Separarea procesării evenimentelor în module independente și utilizarea unei arhitecturi nebloccante permit funcționarea simultană și eficientă a tuturor componentelor aplicației.

Structura generală a aplicației permite extinderea facilă a funcționalităților și integrarea unor mecanisme suplimentare de control inteligent al traficului.

6. CONCLUZII

În cadrul acestui proiect a fost realizată implementarea unui sistem inteligent de semaforizare utilizând platforma STM32 NUCLEO-L476RG și senzori IR pentru detecția traficului. Sistemul dezvoltat permite controlul adaptiv al timpilor de semaforizare în funcție de densitatea traficului detectat în intersecție.

Aplicația a fost implementată utilizând o arhitectură modulară și o mașină de stări finite pentru controlul succesiunii semafoarelor. Utilizarea întreruperilor externe EXTI pentru gestionarea senzorilor a permis detectarea rapidă și eficientă a vehiculelor fără utilizarea unei metode de polling continuu.

Un element important al proiectului îl reprezintă algoritmul adaptiv implementat pentru ajustarea duratei semnalului verde. Acesta modifică dinamic timpul de traversare în funcție de traficul detectat și de existența cozilor de vehicule. De asemenea, a fost implementat un mecanism de prevenire a starvation-ului pentru menținerea unui comportament echilibrat al intersecției.

Monitorizarea funcționării sistemului prin interfața UART a permis verificarea în timp real a funcționării algoritmilor implementați și facilitarea procesului de depanare.

În urma testării aplicației s-a observat funcționarea corectă a tuturor componentelor sistemului: detectarea vehiculelor, schimbarea stărilor semafoarelor, adaptarea timpului de verde și transmiterea informațiilor de monitorizare.

Pe parcursul dezvoltării proiectului au apărut mai multe provocări legate de configurarea întreruperilor externe, stabilizarea semnalelor provenite de la senzori și sincronizarea logicii adaptive cu funcționarea semafoarelor. Aceste probleme au fost rezolvate prin implementarea unor mecanisme software de debounce și prin reorganizarea modulară a codului aplicației.

Sistemul realizat poate reprezenta baza pentru dezvoltarea unor aplicații mai complexe de management inteligent al traficului. Printre posibilele îmbunătățiri viitoare se pot menționa: integrarea unor senzori suplimentari, implementarea comunicației wireless între intersecții, utilizarea unor algoritmi avansați de optimizare a traficului sau dezvoltarea unei interfețe grafice pentru monitorizarea sistemului în timp real.

Proiectul a demonstrat posibilitatea realizării unui sistem embedded funcțional și modular pentru controlul inteligent al traficului utilizând resurse hardware și software accesibile.

7. BIBLIOGRAFIE

- [1] STM32 Product Description, <https://botland.store/stm32-nucleo/18800-stm32-nucleo-l476rg-with-stm32l476rgt6-arm-cortex-m4-mcu-5904422364977.html>;
- [2] Tinkercad Electronics Simulation, <https://www.tinkercad.com/dashboard>;
- [3] EasyEDA Schematics, <https://docs.easyeda.com/en/Introduction/Introduction-to-EasyEDA/>;
- [4] STM32 Documentation, https://stm32world.com/wiki/STM32_Official_Documentation#STM32L4xx;
- [5] Hardware Organization, <https://projecthub.arduino.cc/rinme/smart-traffic-management-system-using-iot-and-esp32-df2858>;
- [6] STMicroelectronics, 'Description of STM32L4/L4+ HAL and low-layer drivers', CubeMX, FSM, 2020.